

Question:01

Part A: Short Questions

1. What is the purpose of `WebServer server(80)` ; and what does port 80 represent?

Ans:

This command allows the ESP32 to behave like a small website that users can access over a network. The number 80 is chosen because web browsers automatically use it for standard HTTP communication.

2. Explain the role of `server.on("/", handleRoot)` ; in this program.

Ans:

This instruction connects the home page request of the server to a specific function. When a user opens the ESP32's IP address in a browser, the program executes `handleRoot()` to prepare and display the webpage.

3. Why is `server.handleClient()` ; placed inside the `loop()` function? What will happen if it is removed?

Ans:

Placing this inside `loop()` ensures the ESP32 keeps responding to users again and again. If it is not included, the ESP32 will ignore incoming browser requests and no webpage will be shown.

4. In `handleRoot()`, explain the statement `server.send(200, "text/html", html);`.

Ans:

This line delivers the webpage to the browser by confirming a successful request, defining the type of data being sent, and transferring the actual HTML content. Without it, the browser would receive an empty response.

5. What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside `handleRoot()`?

Ans:

Using the last measured values simply shows stored sensor data, whereas a fresh reading makes the ESP32 read the sensor again to get the most recent temperature and humidity values.

Part B: Long Question

1. ESP32 Wi-Fi Connection and IP Assignment

Ans:

- The ESP32 connects to a Wi-Fi network inside the `setup()` function using `WiFi.begin(ssid, password)`.
- It keeps trying to connect until the connection is successful, which is checked using `WiFi.status()`.

- After connecting, the router automatically assigns an IP address to the ESP32 using DHCP.
- This IP address can be checked in the program using `WiFi.localIP()`.

In simple terms: The Wi-Fi connection gives the ESP32 an address on the network so other devices can access it like a small web server.

2. Web Server Initialization and Request Handling

Ans:

- A web server is created on the ESP32 using `WebServer server(80)`, where port 80 is the standard port used for web pages.
- The line `server.on("/", handleRoot)`; assigns a function to handle requests made to the main page of the server.
- Inside the `loop()` function, `server.handleClient()`; keeps listening for incoming browser requests and sends responses accordingly.

3. Button-Based Sensor Reading and OLED Update

Ans:

- The button is configured in `INPUT_PULLUP` mode, so it stays HIGH normally and becomes LOW when pressed.
- Button press is detected using a change in state from HIGH to LOW.
- A short delay is added to prevent false triggers caused by button bouncing.
- When the button is pressed, the DHT22 sensor is read to get temperature and humidity values.
- The OLED display is then refreshed to show the new readings.

- These values are also saved in variables so they can be shown on the web page.

4. Dynamic HTML Webpage Generation

Ans:

- The `handleRoot()` function dynamically creates an HTML page using string operations.
- The webpage displays the current temperature and humidity values along with instructions for updating them.
- The completed HTML page is sent to the browser using `server.send(200, "text/html", html);`.
- Any device connected to the same network can open the ESP32's IP address and view this page.

5. Purpose of Meta Refresh in the Webpage

Ans:

- The meta refresh tag is placed inside the HTML header to automatically reload the webpage after a fixed time.
- In this case, the page refreshes every 5 seconds.

- This allows users to see updated sensor readings automatically without refreshing the page manually.
- Without this feature, the webpage would only display the values that were available when it was first opened.

6. Common Issues in ESP32 Webserver Projects and Solutions Projects and Solutions

Ans:

- **Webpage not opening:**

This usually happens if the ESP32 is not connected to Wi-Fi or

`server.handleClient()` is missing from the `loop()`.

Solution: Check Wi-Fi credentials and make sure `server.handleClient()` is continuously running.

- **ESP32 IP address not reachable:**

The IP may change after restarting the ESP32 or the device may be on a different network.

Solution: Always check the current IP using `WiFi.localIP()` and ensure both devices are on the same network.

- **Sensor data not showing on the webpage:**

This can occur if the sensor reading fails or the values are not updated properly.

Solution: Confirm correct sensor wiring, correct DHT type selection, and successful sensor initialization.

Question:02

Part-A: Short Questions

1. What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?

Ans:

The Blynk Template ID acts as a reference that connects the ESP32 to a particular project created on the Blynk Cloud. It must be the same as the cloud template ID so the device can correctly load the assigned dashboard, datastreams, and virtual pin configuration.

2. Differentiate between Blynk Template ID and Blynk Auth Token.

Ans:

The Template ID represents the overall project structure on the Blynk Cloud, whereas the Auth Token is a unique security key that allows an individual ESP32 device to log in and communicate with that project.

3. Why does using DHT22 code with a DHT11 sensor produce incorrect readings? Mention one key difference between the two sensors.

Ans:

DHT11 and DHT22 sensors handle and transmit data differently. One major

difference is that the DHT22 provides more precise readings with decimal values, while the DHT11 outputs whole numbers only. Using DHT22 code with a DHT11 leads to wrong data interpretation.

4. What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?

Ans:

Virtual Pins are logical channels used by Blynk to exchange data between the ESP32 and the cloud. They are preferred because they are independent of actual hardware pins, making data handling more flexible and easier to manage in cloud-based applications.

5. What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?

Ans:

BlynkTimer is used to schedule tasks without stopping the rest of the program. Unlike `delay()`, it allows the ESP32 to remain responsive, maintain a stable cloud connection, and continuously run background services like `Blynk.run()`.

Part-B: Long Question

1. Creation of Blynk Template and Datastreams

First, a Template is created on the Blynk Cloud platform.

The template defines how data will be displayed in the Blynk mobile or web dashboard.

- Widgets (such as gauges or labels) are added to the dashboard.
- Datastreams are created and linked to Virtual Pins:
 - Example: **V0, V1**
- Each datastream defines:
 - Data type (float / integer)
 - Min–max range
 - Update mode (push or periodic)

2. Role of Template ID, Template Name, and Auth Token

- **Template ID**

Identifies which cloud template the ESP32 belongs to. It ensures the device connects to the correct dashboard and datastreams.

- **Template Name**

A human-readable name for the project, mainly used for identification and organization.

- **Auth Token**

A unique security key generated for each device. It authenticates the ESP32 with the Blynk Cloud and allows data exchange.

All three must be correctly defined in the code; otherwise, the device will fail to connect.

3. Sensor Configuration Issues (DHT11 vs DHT22)

DHT11 and DHT22 sensors are **not interchangeable in code**.

- **DHT11**
 - Lower accuracy
 - Integer-only temperature and humidity values
 - Smaller operating range
- **DHT22**
 - Higher accuracy
 - Provides decimal values
 - Wider temperature and humidity range

If DHT22 code is used with a DHT11 sensor (or vice versa), the ESP32 misinterprets sensor data, resulting in **incorrect or NaN readings**.

4. Sending Data

After reading the sensor values, data is sent to the Blynk Cloud using:

```
Blynk.virtualWrite(V0, temperature);  
Blynk.virtualWrite(V1, humidity);
```

- Virtual pins act as a bridge between the ESP32 and Blynk dashboard widgets.
- This method sends sensor data directly to the cloud without relying on physical GPIO pins.
- The dashboard updates in real time whenever new data is pushed.

5. Common Problems and Their Solutions

Wi-Fi connection issues usually happen due to wrong credentials—recheck SSID and password.

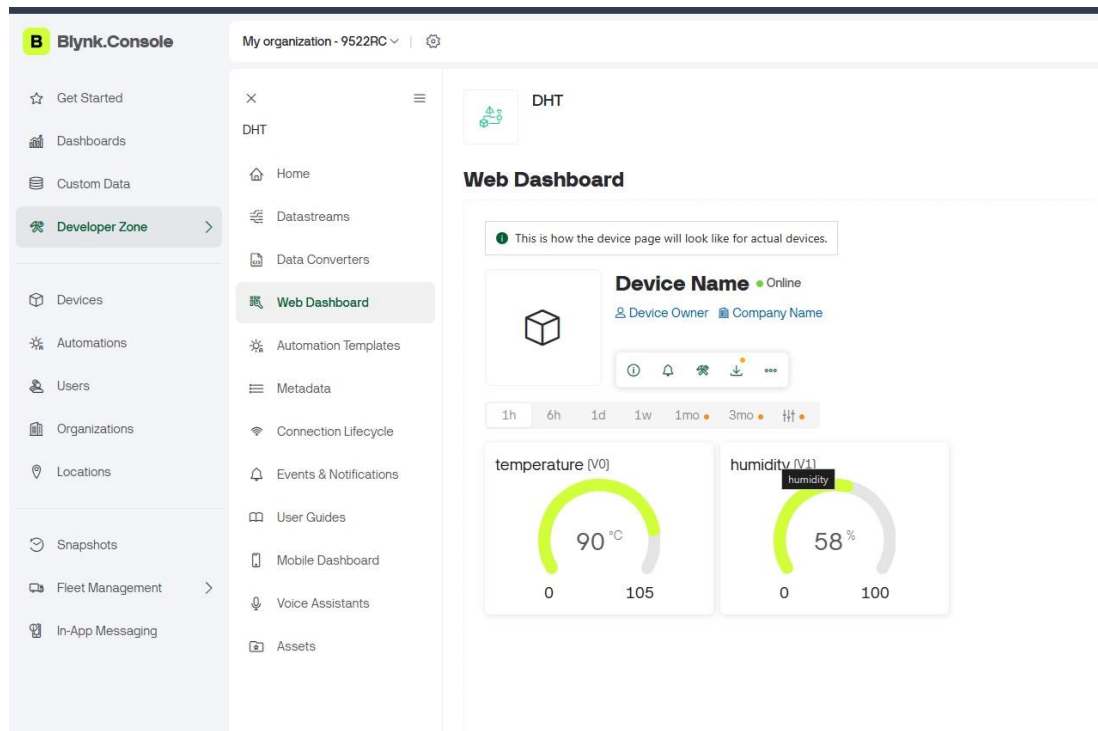
If the web page does not open, `server.handleClient()` is likely missing from `loop()`.

Wrong sensor readings occur when the incorrect DHT type is selected—match DHT11/DHT22 properly.

OLED display problems are mostly due to wrong I2C address or wiring errors.

Using `delay()` can freeze the ESP32—use timers instead for smooth operation.

Cloud Based:



Mobile Based:

